

SIDHARTH SREERAM

Canadian Citizen | TN Visa Eligible (USMCA Engineer)

(416) 564-8590 | sidharthsreeram@gmail.com | sidharth-s.com | linkedin.com/in/sidharthsreeram | github.com/Sidharth-82

University of Waterloo Mechatronics Engineering graduate building robot learning and perception systems in C++ and Python. Seven positions across embedded firmware, test automation, backend APIs, and VLA research.

TECHNICAL SKILLS

Programming Languages: Python, C, C++, MATLAB, SQL, Bash, Ladder Logic

Frameworks & Libraries: PyTorch, NumPy, OpenCV, Boto3, Pandas, Flask, ROS2, Nav2, Gazebo

Software: REST API, Backend Services, Cloud Computing, Database Management, Automated Testing

AI & ML: Reinforcement Learning, Multimodal Data Curation, Synthetic Data, Data Pipelines, VLA, VLM

Robotics: Object Detection, Segmentation, SLAM, RGB-D, LiDAR, Point Cloud, Sensor Fusion, Path Planning

Development: Linux, AWS, Claude Code, Git, LaTeX, Markdown, JSON, VSCode, Docker, Jira, Agile

EDUCATION

University of Waterloo | *B.ASc in Mechatronics & Artificial Intelligence* | *GPA: 3.8* 2021 – 2026

- **Relevant Courses:** Multivariable Control Systems, Control Applications, Cooperative and Adaptive Algorithms, Machine Learning, Autonomous Mobile Robotics, Autonomous Vehicles

EXPERIENCE

VLA Research Assistant - UW WISE Lab (Research) | *Python, PyTorch* Jan 2026 – Apr 2026

- Researched and developed **Python** logic to organize visual and text data into structured VLM prompt format.
- Aggregated NuScenes-QA, NuCaptions, and NuX datasets with multi-sensor metadata by sample token.
- Reconstructed the **OpenDriveVLA** Stage-2 prompt spec, fusing camera, LiDAR, and CAN-bus ego state.
- Generated **728K** JSONL instruction-tuning samples for camera-vs-LiDAR VLA reliability research.

Software Analyst - Tamaki Controls (Coop) | *Python, Java, SQL, Ignition* May 2025 – Aug 2025

- Designed an **API** response schema improving cross-service communication between **Java** and **Python** stack.
- Built 6 customer-facing **REST** endpoints importing and exporting UTF-8 encoded CSV/JSON data.
- Wrote **SQL** queries aggregating source data from the database backing each export endpoint.
- Updated **API documentation** with input/output descriptions and example Python client scripts.

Platform Software Engineer - Ford (Coop) | *Python, C++, GCov* Sept 2024 – Dec 2024

- Developed a **Python CLI** for the AUTOSAR State Management API in a containerized Linux environment.
- Automated 6-stage state transitions, replacing stage-by-stage manual navigation with one command.
- Drove CLI adoption across **3 engineering teams** for state control in testing and integration.
- Created C++ test suites with GCov, raising line coverage **45% to 94%** and condition coverage **50% to 85%**.

Controls System Engineer - Brock Solutions (Coop) | *Python, SQL, Ignition* Jan 2024 – Apr 2024

- Built **Ignition SCADA HMI** screens visualizing live coil crane, acid bath, and pump operations at a steel facility.
- Simulated PLC outputs across a **27-PLC** system to validate HMI designs pre-commissioning.
- Collaborated in an **Agile** workflow using **Azure DevOps** to manage and track user stories and story points.

Product Developer - Gen AI Ventures (Coop) | *Flutter, Database Management* May 2023 – Aug 2023

- Collaborated in startup environment with founder and CPO to architect a social matching application using Flutter.
- Built responsive **Flutter** front-end with a reusable widget/component structure for adaptive UI across screens.
- Structured the Firebase backend: OAuth for sessions, Firestore for database, Cloud Functions for serverless logic.

Process Engineering Student - Matcor (Coop) | *Data Management, Excel* Sept 2022 – Dec 2022

- Gathered and structured use-case data of tooling across plant operations to evaluate equipment against project specs.
- Built Excel analyses turning the data into recommendations: commission new tooling or reuse existing machines.
- Conducted observational studies to obtain workflow data, resulting in enhancements to process efficiency and outputs.

Embedded Software Developer - Peraso (Coop) | *C/C++, Automated Testing* Jan 2022 – Apr 2022

- Developed and refactored embedded C++ firmware on an MCU managing serial flash for pre-production 5G silicon.
- Owned the flash-specific parser converting hex constant commands into executable bit sequences via bit-shifting.
- Led root cause analysis finding duplicate parsing functions; cutting write latency 5ms→1.3ms and read 2ms→1ms.

Cloud-Native ADAS Perception Stack | *Python, C++, ROS2, YOLOv11, CARLA, LiDAR, AWS EC2/S3, Docker*

- **Purpose:** Measuring whether cloud-served perception can meet closed-loop highway driving, and where it breaks.
- Generated an 8,400-frame, 14,396-object **KITTI** dataset in **CARLA** (RGB, depth, LiDAR, ego pose).
- Trained a **YOLOv11** vehicle detector against a per-class recall gate; traced a failing van class to far-range, low-diversity data (145 labels, 13 vehicles), fixed with a targeted 387-frame capture and 960px retrain: van recall **0.78→0.86**, macro mAP **0.84→0.89**.
- Profiled inference at **8.9 ms p50 / 11.2 ms p99** (A10G); flat across FP16 and input size, so overhead-bound, not compute.
- Ran headless CARLA on **EC2** spot GPUs with dockerized jobs, a custom AMI (~2 min relaunch), and a bounded-memory **S3** loader.
- **In progress:** **ROS2** C++ closed-loop highway cruise control (YOLO + LiDAR 3D boxes, EKF tracking, lane keeping) targeting a Jetson Orin Nano, scored against a ground-truth perception run to measure the detector's cost.

VLA Policy Eval Layer | *Python, PyTorch, OpenVLA, LIBERO, MuJoCo, Docker, AWS EC2, Statistics*

- **Purpose:** Harness measuring where a **vision-language-action** policy (OpenVLA-7B) breaks under instruction paraphrase.
- Reproduced the published **OpenVLA/LIBERO** baseline (**83.6%** vs 84.7%, n=500) inside a ± 5 pp gate fixed before the run.
- Built a seeded, **Dockerized** runner and validated it episode-by-episode against the reference script: **500/500** identical.
- Ran a **2,000-episode**, 32 GPU-hour paraphrase sweep; paired **McNemar** tests found synonym swaps not significant while sentence-structure rewrites cut success by **11.6–12.6pp**.
- Separated sensitivity from degradation: word swaps flipped **23%** of episodes both ways with no significant net change.
- **In progress:** automatic failure taxonomy, Wilson-interval slice report, and a bf16 vs. 4-bit policy regression report.

Self-Balancing Pendulum RL Control System | *C++, Reinforcement Learning, LibTorch, MuJoCo, Python, CMake*

- Built a C++ simulator, controller, and trainer for a motorized cart on a 2m rail balancing a free-swinging pole, where the only available control is accelerating the cart.
- Derived and implemented the pendulum physics from the equations of motion, running at 500Hz and modeling real motor force limits, sensor resolution and delay, and lost motor steps.
- Wrote the reinforcement learning algorithm (PPO) in C++ on LibTorch, training the neural network controller entirely on a laptop CPU.
- Trained a single policy that swings the pole upright from hanging and holds it there, reaching 96% of a perfect score, upright in 0.31 seconds and held within 0.015 degrees.
- Visualized network controller on MuJoCo, a physics engine, to verify reliability of the controller in real-world setting.

Machine Vision Path Following Robot | *Python, OpenCV, YOLO, Pure Pursuit, Raspberry Pi 5*

- Built a real-time **OpenCV** vision pipeline on embedded hardware for colour-based detection and tracking.
- Calibrated **RGB** camera intrinsics with chessboard targets and corrected lens distortion for accurate frames.
- Applied a homography perspective transform to map camera view to real-world top-down coordinates.
- Tuned HSV thresholding, morphological filtering, and contour extraction to isolate the path under varying lighting.
- Computed a look-ahead point via **Pure Pursuit**, deriving steering curvature regulated by a **PID** loop.
- Proposed and tested YOLOv8 object detection model to notify upon locating the target.

UWB Tag & Anchor Tracking System (Capstone) | *C++, ESP32-S2, PCBA, TDoA, RF Design, Localization*

- Developed **TDoA** localization using **weighted least squares** with **Firefly** optimization, reaching roughly **30 cm** 3-D accuracy in deployment.
- Implemented wireless clock synchronization for nanosecond-level timestamp reliability across the network.
- Designed and fabricated anchor PCBs integrating the **DW3110 UWB** transceiver and **ESP32-S2** MCU.
- Routed controlled-impedance RF traces, then brought up and debugged boards for reliable high-frequency signaling.